

Code Explanation

Testing Delta Live Tables Transformations

This test file provides a simplified approach to test the transformations applied in a Delta Live Tables (DLT) pipeline. The test class, `TestDeltaLiveTables`, contains methods to set up a test environment, create sample data, and test various transformations.

Overview of the Code

The code defines a test class that inherits from `unittest.TestCase`. It sets up a `SparkSession`, creates temporary directories for test data, defines schemas for customer and order data, and creates sample DataFrames. The test method, `test\_dlt\_transformations`, tests various transformations that would be applied in a DLT pipeline.

Step 1: Setting Up the Test Environment

This step creates a `SparkSession` with the necessary configurations for Delta Lake and sets up temporary directories for test data.

```
cls.spark = SparkSession.builder
    .appName("TestDeltaLiveTables")
    .master("local[*]")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .getOrCreate()
```

Step 2: Defining Schemas for Customer and Order Data

This step defines the schemas for customer and order data using `StructType` and `StructField`.

```
cls.customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

cls.order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])
```

Step 3: Creating Sample Data and DataFrames

This step creates sample customer and order data, and then creates DataFrames using the defined schemas.

```
customer_data = [
    ("C001", "John Doe", "john@example.com", "North"),
    ("C002", "Jane Smith", "jane@example.com", "South"),
    ("C003", "Bob Johnson", "bob@example.com", "East"),
    ("C004", None, "alice@example.com", "West"),
    ("C001", "John Doe", "john@example.com", "North") # Duplicate
]

order_data = [
    ("0001", "Laptop", 1000.0, 2, datetime.date(2023, 1, 15), "C001"),
    ("0002", "Phone", 500.0, 1, datetime.date(2023, 1, 20), "C002"),
    ("0003", "Tablet", 300.0, 3, datetime.date(2023, 1, 25), "C003"),
    ("0004", "Headphones", 100.0, None, datetime.date(2023, 1, 30), "C001"),
    ("0002", "Phone", 500.0, 1, datetime.date(2023, 1, 20), "C002") # Duplicate
]

cls.customer_df = cls.spark.createDataFrame(customer_data, cls.customer_schema)
cls.order_df = cls.spark.createDataFrame(order_data, cls.order_schema)
```

Step 4: Writing Sample Data to Temporary Files

This step writes the sample DataFrames to temporary CSV files.

```
cls.customer_df.write.format("csv").option("header", "true").mode("overwrite").save(cls.customer_data_path)
cls.order_df.write.format("csv").option("header", "true").mode("overwrite").save(cls.order_data_path)
```

Step 5: Testing DLT Transformations

This step tests various transformations that would be applied in a DLT pipeline, including reading data from temporary files, cleaning and transforming data, and joining customer and order data.

```
def test_dlt_transformations(self):
    # Test customer_bronze transformation
    customer_bronze = self.spark.read.format("csv").option("header", "true").load(self.customer_data_path)
    self.assertEqual(customer_bronze.count(), 5)

    # Test order_bronze transformation
    order_bronze = self.spark.read.format("csv").option("header", "true").load(self.order_data_path)
    self.assertEqual(order_bronze.count(), 5)
```

```

    # Test customer_silver transformation (clean customer data)
    customer_silver = customer_bronze.dropDuplicates().na.drop()
    self.assertEqual(customer_silver.count(), 3) # 5 original - 1 duplicate - 1 with null

    # Test order_silver transformation (clean order data and add TotalAmount)
    from pyspark.sql.functions import col
    order_silver = order_bronze.dropDuplicates().na.drop()
    .withColumn("PricePerUnit", col("PricePerUnit").cast("double"))
    .withColumn("Qty", col("Qty").cast("integer"))
    .withColumn("Date", col("Date").cast("date"))
    .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))

    self.assertEqual(order_silver.count(), 3) # 5 original - 1 duplicate - 1 with null

    # Test ordersummary transformation (join customer and order data)
    ordersummary = customer_silver.join(order_silver, "CustId")
    self.assertTrue(ordersummary.count() > 0)

    # Test customeraggregatespend transformation (aggregate TotalAmount by Name and Date)
    from pyspark.sql.functions import sum as sum_
    customeraggregatespend = ordersummary.groupBy("Name", "Date").agg(
        sum_("TotalAmount").alias("TotalAmount"))
    self.assertTrue(customeraggregatespend.count() > 0)

```